Trabajo Final: Caso de Estudio Algorítmico

Optimización de Tareas en Servidores

Estudiantes
Simon Correa Bravo
David Hernández Mejia
Juan P. Tafur Corral

Análisis y Diseño de Algoritmos

Universidad de Medellín
2026-2



Tabla de Contenidos

1. Introducción

Se empleo un algoritmo simple para desarrollarlo con diferentes técnicas de programación tales como: Fuerza Bruta, Greedy, Backtracking, Divide y Vencerás, Programación Dinámica. Con el objetivo de identificar cual es el mejor método para solucionar el algoritmo, se desarrollará cada técnica y se identificaran ciertos datos para concluir cual es el mejor.

2. Definición del Problema

El problema describe un sistema de cómputo compuesto por cierta cantidad de servidores los cuales deben atender una cantidad finita de tareas con las siguientes características: tiempos de ejecución, prioridad y dependencias. Se necesita minimizar el tiempo total de ejecución a la hora de atender estas tareas.

2.1 Entradas y salidas

Entradas:

- num_tareas: Número total de tareas
- num_servidores: Número total de servidores

Salidas:

- tiempo_completar_tareas: tiempo total mínimo para completar todas las tareas
- asignacion_optima: una tupla para mapear en que servidor queda asignada cada tarea

Restricciones:

- Cada tarea debe asignarse a exactamente a un servidor
- Las tareas no se pueden interrumpir ni dividir
- Todos los servidores procesan las tareas a la misma velocidad

2.2 Datos de Prueba

Antes que nada, cabe aclarar que los datos de prueba pueden cambiar un poco dependiendo de la eficiencia de problema, lo que exige poner un problema con un mayor volumen.

Caso pequeño (4 tareas, 2 servidores)

Tareas	Tiempo	Dependencias
A	5	-
B	3	A
C	4	A
D	2	B, C

Caso mediano (9 tareas, 3 servidores)

Tareas	Tiempo	Dependencias
A	8	-
B	13	-
C	5	A
D	5	A
E	4	B, C
F	6	C
G	8	E, A
H	10	D, E, F
I	7	G

Caso grande (10 tareas, 3 servidores)

Tareas	Tiempo	Dependencias
A	8	-
B	13	-
C	5	A
D	5	A
E	4	B, C
F	6	C
G	8	E, A
H	10	D, E, F
I	7	G
J	9	B

3. Fuerza Bruta

3.1 Definición

La fuerza bruta consiste en probar todas las soluciones posibles y quedarse con la mejor. En este problema, eso significa generar todas las formas válidas de ordenar y asignar las tareas a los servidores, calcular el tiempo total de cada una y elegir la que termine antes.

3.2 Implementación

Para la implementación de esta técnica, lo primero que se definió fueron las librerías que se usaron durante el desarrollo. De la librería **itertools** se importaron dos funciones: permutations, que genera todos los órdenes posibles de las tareas, y product, que genera todas las formas de asignar las tareas a los servidores. También se usó la librería **time**, específicamente la función perf_counter, para medir con precisión el tiempo de ejecución del algoritmo.

Luego se creó la clase **Tarea**, representa cada tarea del problema. Cada tarea tiene un identificador, un tiempo de ejecución y una lista con las tareas de las que depende. Esta clase solo almacena información, por lo que no tiene impacto en el rendimiento del algoritmo.

El primer paso de este algoritmo es la función **definir_dependencias.** Esta función recorre todas las permutaciones posibles de las tareas, para cada una verifica que ninguna tarea aparezca antes que otra de la que depende. Se construye un diccionario con la posición de cada tarea dentro de la permutación. Así comprobar si una dependencia se cumple solo requiere comparar dos opciones. Si se encuentra una dependencia que no se cumple, se descarta y se pasa a la siguiente. Al final, la función devuelve una lista con los órdenes válidos.

Finalmente, la función principal **fuerza_bruta,** recibe las tareas y el numero de servidores. Para cada orden válido, prueba todas las formas posibles de asignar las tareas a los servidores. En cada combinación se simula la ejecución: una tarea solo puede empezar cuando su servidor está libre y cuando todas sus dependencias han terminado. Al final, el tiempo total de esa combinación corresponde al momento en que termina la última tarea y se conserva la que tenga el menor tiempo total.

Finalmente, se definieron tres casos de prueba de tamaño: uno pequeño de 4 tareas y 2 servidores, uno mediano de 9 tareas y 3 servidores, y uno grande de 10 tareas y 3 servidores. Los casos mediano y grande se construyeron a partir de pruebas con diferentes datos forma que el grande se diferencia al mediano por un buen trecho, lo que permite observar cómo afecta al tiempo de ejecución el aumento de una sola tarea.

4. Complejidad Temporal – Espacial

Para facilitar el proceso de sacar la complejidad del algoritmo, se separó el código en las diferentes funciones, seleccionando como complejidad la función principal y con más complejidad dentro de este. Haciendo que sea mucho más sencillo de comprender.

4.1 Complejidad Temporal

Primero calculamos la complejidad temporal de la clase **Tarea.** La clase **Tarea** solo almacena datos; __init__ solo hace tres asignaciones, cada una vale $O(1)$ Y el __rep__ convierte el id en texto. Técnicamente depende de la longitud del id, se considera $O(1)$ siendo esta la complejidad total de esta porción de código.

Para la función **definir_dependencias** la complejidad es $O(n! * n^2)$. La función recorre las n! permutaciones de las tareas. Para cada una construye un diccionario de posiciones en $O(n)$ y luego revisa todas las dependencias dentro de este $O(n)$. Lo que equivale a $O(n! * n^2)$.

Ahora la función principal **fuerza_bruta** dentro del algoritmo tiene una complejidad de $O(n! * m^n * n^2)$ El factor $n!$ proviene de revisar todos los órdenes posibles de las tareas, m^n de probar todas las formas de asignarlas a los servidores, y n^2 del costo de simular cada combinación revisando las dependencias. Al final nos da una complejidad de $O(n! * m^n * n^2)$ y al ser la función principal, con una complejidad mayor, nos quedamos con esta para representar todo el algoritmo.

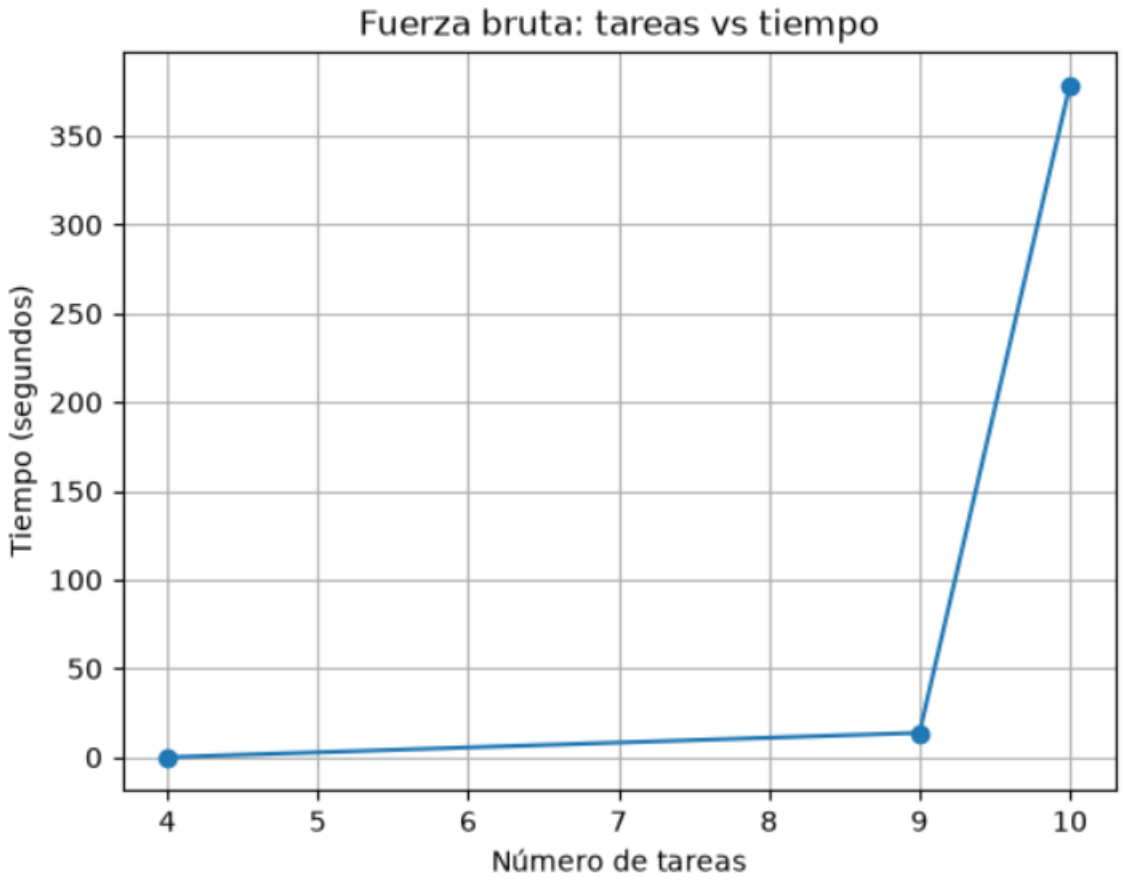
4.2 Complejidad Espacial

La complejidad espacial es la cantidad de almacenamiento que se necesita en base a una entrada. Para calcular la complejidad de la clase **Tarea** identificamos que cada objeto guarda tres cosas, id, tiempo, prioridad y una referencia a la lista de dependencias. Lo que se guarda son los tres atributos de tarea, cada uno con complejidad $O(1)$.

En la función **definir_dependencias** se guarda permutations, posición, es_valida y validas que es la lista resultante, esta última es realmente la única que se acumula y calculamos la complejidad identificando que tiene hasta $n!$ Permutaciones de tamaño n lo que nos da un total de $O(n! * n)$.

Para la función **fuerza_bruta** directamente solo se acumula la lista proveniente del método anterior (validos) de tamaño $O(n! * n)$ y con esta última podemos concluir que el algoritmo en su totalidad es $O(n! * n)$.

5. Grafica



La grafica muestra que, con pocas tareas, la fuerza bruta es casi instantánea. En cambio, al pasar de 9 a 10 tarreas, el tiempo sube de unos 14 segundos a casi 340, es decir, se multiplica por 27. Esto ocurre porque el algoritmo prueba todas las combinaciones posibles, y cada tarea nueva multiplica la cantidad de combinaciones en lugar de sumarlas. Este comportamiento coincide con su complejidad $O(n! * m^n * n^2)$ y se puede concluir que esta técnica solo es viable para problemas pequeños.